LECTURE 1: INTRODUCTION

1.1 Course & Programming Overview

Computer Program: A predefined set of instructions followed by a computer to perform specific tasks.

Compilation Process: High-level programming language code is translated by a **compiler** into machine language (low-level machine instructions/object code).

Program Development Cycle:

1. Analysis & Design Phase: Define the problem, check logic, and propose an overall solution.

2. Write:Type C++ source code into a source file.

3. Compile: Translate C++ source code into object code stored in an object file.

4. Link: Combine object files with C++ run-time libraries to create an executable file.

5. Test: Execute the program to identify errors.

6. Debug: Locate and fix compile-time syntax errors, run-time errors, or logic errors (bugs).

1.2 Brief History of C++

1972: Dennis Ritchie developed the C language at AT&T Bell Labs.

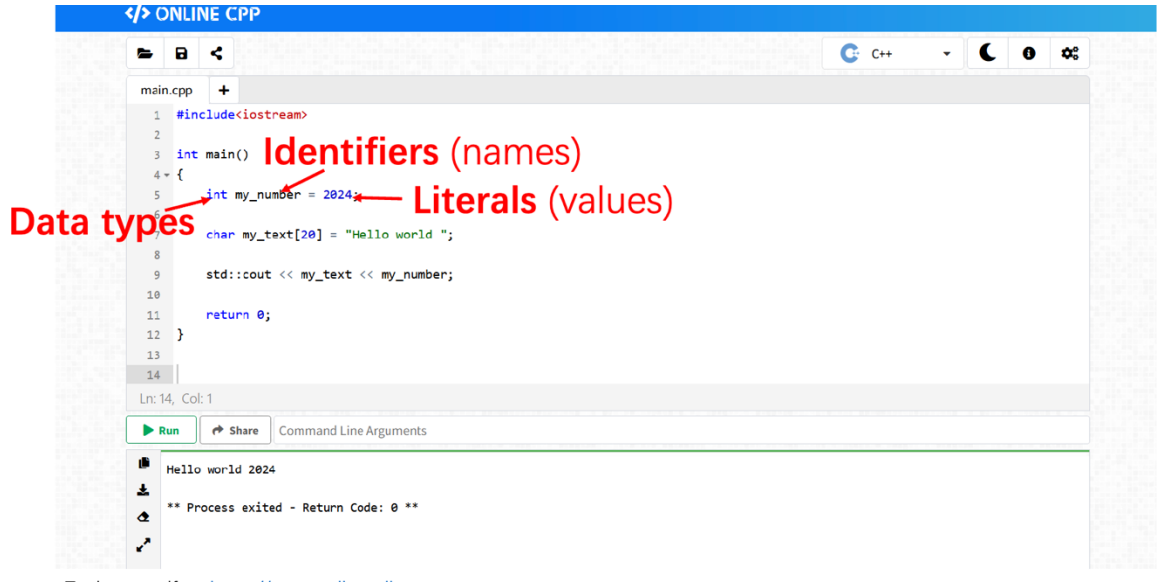
1980:Bjarne Stroustrup created "C with Classes".

1983:Bjarne Stroustrup officially developed C++ at AT&T Bell Labs.

1985: First commercial version released.

1997:ANSI/ISO C++ standard finalized.

LECTURE 2: BASIC SYNTAX OF C++



2.1 Identifiers

Definition: Names used to identify variables, constants, functions, or arrays.

Naming Rules:

1. Sequence of letters, digits, and underscores (`_`).

2. Must NOT start with a digit.

3. Case-sensitive (For example; counter not equal to cOuNtEr).

4. Cannot use C++ reserved keywords (e.g., int, class, return, switc).

2.2 Fundamental Data Types & Memory Sizes

Memory allocated depends on system architecture and data type (checked using “sizeof()” operator):

Category	Type	Typical Size	Description
Boolean	bool	1 Byte	Stores “true” (1) or “false” (0)
Character	char	1 Byte	Stores single ASCII characters
Integer	short int	2 Bytes	Signed/unsigned integer
	int	4 Bytes	Standard signed/unsigned integer

	long int	8 Bytes	Extended integer
	long long int	8 Bytes	Large integer
Floating Point	float	4 Bytes	Single precision floating-point
	double	8 Bytes	Double precision floating-point
	long double	16 Bytes	High-precision floating-point

2.3 Literals & Bases

Integer Bases:

Decimal (Base 10): No prefix (e.g., `42`).

Octal (Base 8):Prefix `0` (e.g., `052` = 5 * 8 + 2 = 42).

Hexadecimal (Base 16):Prefix `0x` or `0X` (e.g., `0x2A` = 2 * 16 + 10 = 42).

Binary (Base 2):Prefix `0b` or `0B` (e.g., `0b101010` = 42$).

2.4 Variables & Uninitialized Variables

Declaration: `int my_number;`

Assignment: `my_number = 2024;`

Uninitialized Variables: C++ does **not** automatically initialize local variables to zero; uninitialized variables contain whatever arbitrary **garbage value** currently resides in that memory location.

2.5 Data Type Conversion

Automatic (Implicit): Converts lower data types to higher data types automatically without data loss (e.g., `int` to `double`). Converting higher to lower causes **data loss**.

Explicit (C-style Casting): Manually forced type casting: `a = (int)b;`

ASCII Chart

main.cpp

```cpp
#include <iostream>

int main()
{
    for (char c='A'; c <= '^'; ++c)
        std::cout << c << ' ';
}
```

Ln: 1, Col: 20

Run | Share | Command Line Arguments

A B C D E F G H I J K L M N O P Q R S T U V W X Y Z [\ ] ^

Code	Symbol	Code	Symbol	Code	Symbol	Code	Symbol
0	NUL (null)	32	(space)	64	@	96	`
1	SOH (start of header)	33	!	65	A	97	a
2	STX (start of text)	34	"	66	B	98	b
3	ETX (end of text)	35	#	67	C	99	c
4	EOT (end of transmission)	36	$	68	D	100	d
5	ENQ (enquiry)	37	%	69	E	101	e
6	ACK (acknowledge)	38	&	70	F	102	f
7	BEL (bell)	39	'	71	G	103	g
8	BS (backspace)	40	(	72	H	104	h
9	HT (horizontal tab)	41	)	73	I	105	i
10	LF (line feed/new line)	42	*	74	J	106	j
11	VT (vertical tab)	43	+	75	K	107	k
12	FF (form feed / new page)	44	,	76	L	108	l
13	CR (carriage return)	45	-	77	M	109	m
14	SO (shift out)	46	.	78	N	110	n
15	SI (shift in)	47	/	79	O	111	o
16	DLE (data link escape)	48	0	80	P	112	p
17	DC1 (data control 1)	49	1	81	Q	113	q
18	DC2 (data control 2)	50	2	82	R	114	r
19	DC3 (data control 3)	51	3	83	S	115	s
20	DC4 (data control 4)	52	4	84	T	116	t
21	NAK (negative acknowledge)	53	5	85	U	117	u
22	SYN (synchronous idle)	54	6	86	V	118	v
23	ETB (end of transmission block)	55	7	87	W	119	w
24	CAN (cancel)	56	8	88	X	120	x
25	EM (end of medium)	57	9	89	Y	121	y
26	SUB (substitute)	58	:	90	Z	122	z
27	ESC (escape)	59	;	91	[	123	{
28	FS (file separator)	60	<	92	\	124	|
29	GS (group separator)	61	=	93	]	125	}
30	RS (record separator)	62	>	94	^	126	~
31	US (unit separator)	63	?	95		127	DEL (delete)

LECTURE 3: OPERATORS AND CONSTANTS

3.1 Arithmetic Operators

Basic Operators:

`+` (addition),

`-` (subtraction),

`*` (multiplication),

`/` (division),

`%` (modulus).

Integer vs Floating Division:

`5 / 2` -`2` (Integer division truncates fractional part).

`5.0 / 2.0` $\rightarrow$ `2.5` (Floating-point division).

Modulus Operator (`%`): Operands must be integers. Formula: A % B = A - (A / B) * B. Example: `-10 % 4` will be`-2`.

3.2 Increment & Decrement Operators

Prefix (`++a`, `--a`): Value is modified **before** the surrounding expression is evaluated.

Postfix (`a++`, `a--`):** Original value is evaluated in the expression first, then modified **after.**

3.3 Constants

Avoid magic numbers in code using constants:

1. Macro Constants (`#define`): Preprocessor directive (no type checking, no trailing semicolon).

```
#define PI 3.1415926
```

2. **`const` Variable:** Type-safe constant enforced by compiler.

```
const double PI = 3.1415926;
```

3.4 Operator Precedence & Associativity

1. Postfix (`a++`, `a--`)

2. Prefix (`++a`, `--a`), Unary (`+`, `-`)

3. Multiplicative (`*`, `/`, `%`)

4. Additive (`+`, `-`)

5. Assignment (`=`, `+=`, `-=`, `*=`, `/=`, `%=`) (Associativity: **Right-to-Left**)

Precedence	Operator	Description	Associativity
1	`a::b`	Scope resolution	Left-to-right →
2	`a++` `a--`	Suffix/postfix increment and decrement	
	`type``(a)` *`type`*`{a}`	Functional cast	
	`a()`	Function call	
	`a[]`	Subscript	
	`a.b` `a->b`	Member access	
3	`++a` `--a`	Prefix increment and decrement	Right-to-left ←
	`+a` `-a`	Unary plus and minus	
	`!a` `~a`	Logical NOT and bitwise NOT	
	`(`*`type`*`)a`	C-style cast	
	`*a`	Indirection (dereference)	
	`&a`	Address-of	
	`sizeof`	Size-of[note 1]	
	`co_await`	await-expression (C++20)	
	`new` – `new[]`	Dynamic memory allocation	
	`delete` – `delete[]`	Dynamic memory deallocation	
4	`a.*b` `a->*b`	Pointer-to-member	Left-to-right →
5	`a * b` `a / b` `a % b`	Multiplication, division, and remainder	
6	`a + b` `a - b`	Addition and subtraction	
7	`a << b` `a >> b`	Bitwise left shift and right shift	
8	`a <=> b`	Three-way comparison operator (since C++20)	
9	`a < b` `a <= b` `a > b` `a >= b`	For relational operators < and <= and > and >= respectively	
10	`a == b` `a != b`	For equality operators = and != respectively	
11	`a & b`	Bitwise AND	
12	`a ^ b`	Bitwise XOR (exclusive or)	
13	`a | b`	Bitwise OR (inclusive or)	
14	`a && b`	Logical AND	
15	`a || b`	Logical OR	
16	`a ? b : c`	Ternary conditional[note 2]	Right-to-left ←
	`throw`	throw operator	
	`co_yield`	yield-expression (C++20)	
	`a = b`	Direct assignment (provided by default for C++ classes)	
	`a += b` `a -= b`	Compound assignment by sum and difference	
	`a *= b` `a /= b` `a %= b`	Compound assignment by product, quotient, and remainder	
	`a <<= b` `a >>= b`	Compound assignment by bitwise left shift and right shift	
	`a &= b` `a ^= b` `a |= b`	Compound assignment by bitwise AND, XOR, and OR	
17	`a, b`	Comma	Left-to-right →

LECTURE 4: SIMPLE INPUT/OUTPUT (I/O)

4.1 Stream Objects

-`cout`: Standard output stream associated with the screen, used with insertion operator `<<`.

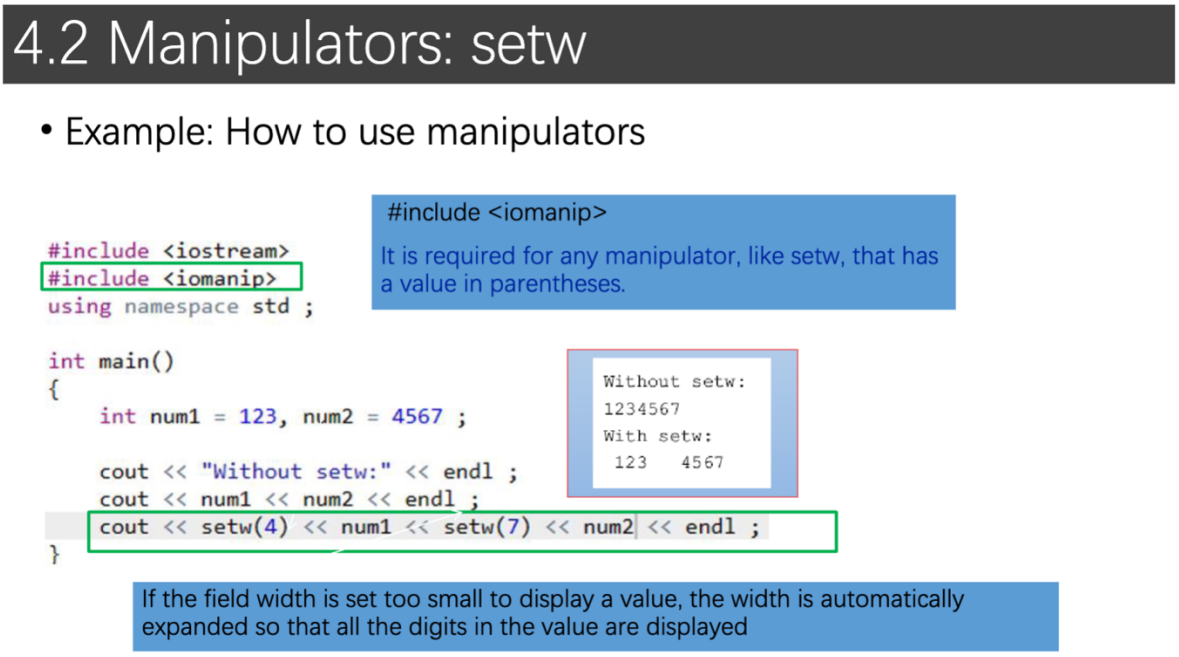
-`cin`: Standard input stream associated with the keyboard, used with extraction operator `>>`.

-`cin >>` stops reading at white-space characters (spaces, tabs, newlines).

4.2 Stream Manipulators (Requires `#include <iomanip>`)

`endl`:Inserts a newline character and flushes output buffer.

`setw(n)`: Sets field width for the next item only.



`setfill(c)`:Sets padding character (persists for subsequent outputs).

- Example: How to use manipulators

```
#include <iostream>
#include <iomanip>
using namespace std ;

int main()
{
    double num = 123.456 ;

    cout << setw(9) << setfill('*') << num << endl ;
    cout << setw(9) << setfill('0') << num << endl ;
    cout << setw(10) << num << endl ;
}
```

```
**123.456
00123.456
000123.456
```

Unlike *setw* ,which applies only to the next data item in the output stream, the *setfill* manipulator remains in effect for all subsequent the fill character remains 0.

`setprecision(n)` : Sets total significant digits (or decimal places when combined with `fixed`).

`fixed` : Formats floating-point output in fixed-point notation.

- Example: How to use manipulators

```
#include <iostream>
#include <iomanip>
using namespace std ;

int main()
{
    double num = 123.45678 ;

    cout << num << endl ;
    cout << setprecision(7) << num << endl ;
    cout << fixed <<  setprecision(2) << num << endl ;
}
```

```
123.457
123.4568
123.46
```

- setprecision: the *total* digits to reserve *before and after* the decimal point (default is 6)
- With fixed preceding, it sets the digits to reserve *after* the decimal point
- Both fixed and setprecision remain in effect for subsequent insertions into the output stream

4.3 Single-Character I/O

`cin.get(ch)` : Reads the next single character, including whitespace characters.

`cout.put(ch)`:Displays a single character to standard output.

`noskipws`:Stream manipulator forcing `cin` to read whitespace rather than skipping it.

- Character Input and Output
  - Using the manipulator ***noskipws***.(no skip whitespaces).

```
std::cout << "Please enter a character: ";
std::cin >> std::noskipws >> ch; // Reads the next character,
                                 // a whitespace character
                                 // may be read

std::cout << "You entered: " << ch << std::endl;
```

```
Please enter a character:
        a
You entered:
```

LECTURE 5: PROGRAMMING CONTROL STRUCTURES

5.1 Relational & Logical Operators

Relational Operators: `>`, `<`, `>=`, `<=`, `==`, `!=`.

Warning: Avoid continuous comparisons like `1 < b < 3`; use logical expressions instead: `(b > 1) && (b < 3)`.

Logical Operators:

-`&&` (AND): Returns `true` only if both conditions are true.

-`||` (OR): Returns `true` if at least one condition is true.

-`!` (NOT): Inverts logical state.

5	a*b a/b a%b	Multiplication, division, and remainder
6	a+b a-b	Addition and subtraction
7	<< >>	Bitwise left shift and right shift
8	<=>	Three-way comparison operator (since C++20)
9	< <= > >=	For relational operators < and ≤ and > and ≥ respectively
10	== !=	For equality operators = and ≠ respectively
11	a&b	Bitwise AND
12	^	Bitwise XOR (exclusive or)
13	|	Bitwise OR (inclusive or)
14	&&	Logical AND
15	||	Logical OR

5.2 Selection Control Structures

Single Selection:

if (condition) { /* statements */ }

Double Selection:

if (condition) { /* statements */ }

else { /* statements */ }

Ternary (Conditional) Operator: Inline shortcut for `if-else` :

max_value = (a > b) ? a : b;

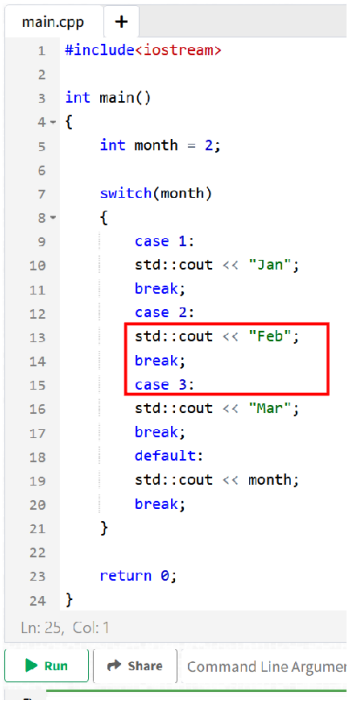
```
int grade;
std::cin >> grade;

if(grade >= 95)
{
    std::cout << "A+"
}
else if(grade >= 90)
{
    std::cout << "A";
}
else if(grade >= 80)
{
    std::cout << "B";
}
else
{
    std::cout << "C";
}
```

Switch Statement: Evaluates an integer or character expression.

```
switch (expression) {
  case val1:
    // statements
    break; // prevents fall-through
  default:
    // default statements
    break;
}
```

Fall-through: If `break` is omitted, execution continues into subsequent cases regardless of their condition.



5.3 Iteration (Loop) Structures

`for` Loop:** Used when the number of iterations is known in advance.

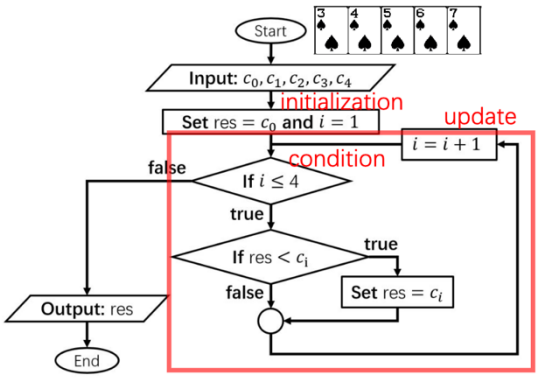
for (initialization; condition; update) { /* body */ }

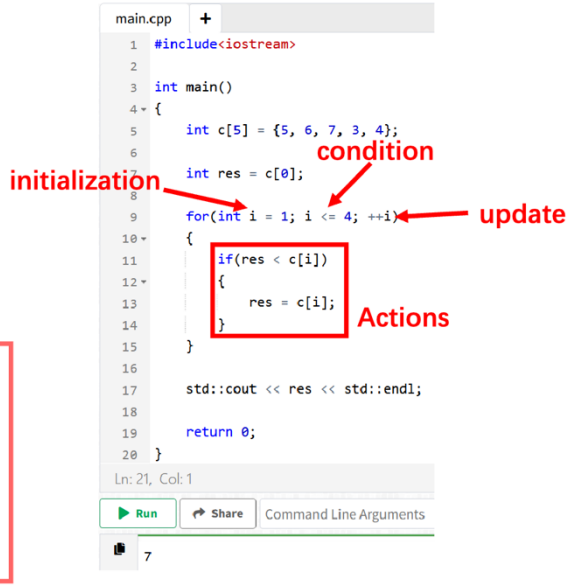
“for” loop

- Syntax

Use semicolons!

```
for (initialization ; condition ; update )
{
      Actions
}
```





`while` Loop:** Evaluates condition **before** executing body (0 or more executions).

while (condition) { /* body */ }

“while” loop

- Syntax

```
while (condition)
{
      Actions
}
```

The while loop simply repeats statement while the condition is true. If after any execution of actions the condition is no longer true, the loop will end and the program continues right after the loop.

```
// custom countdown using while
#include <iostream>
using namespace std;

int main ()
{
  int n = 10;

  while (n>0) {
    cout << n << ", ";
    --n;
  }

  cout << "liftoff!\n";
}
```

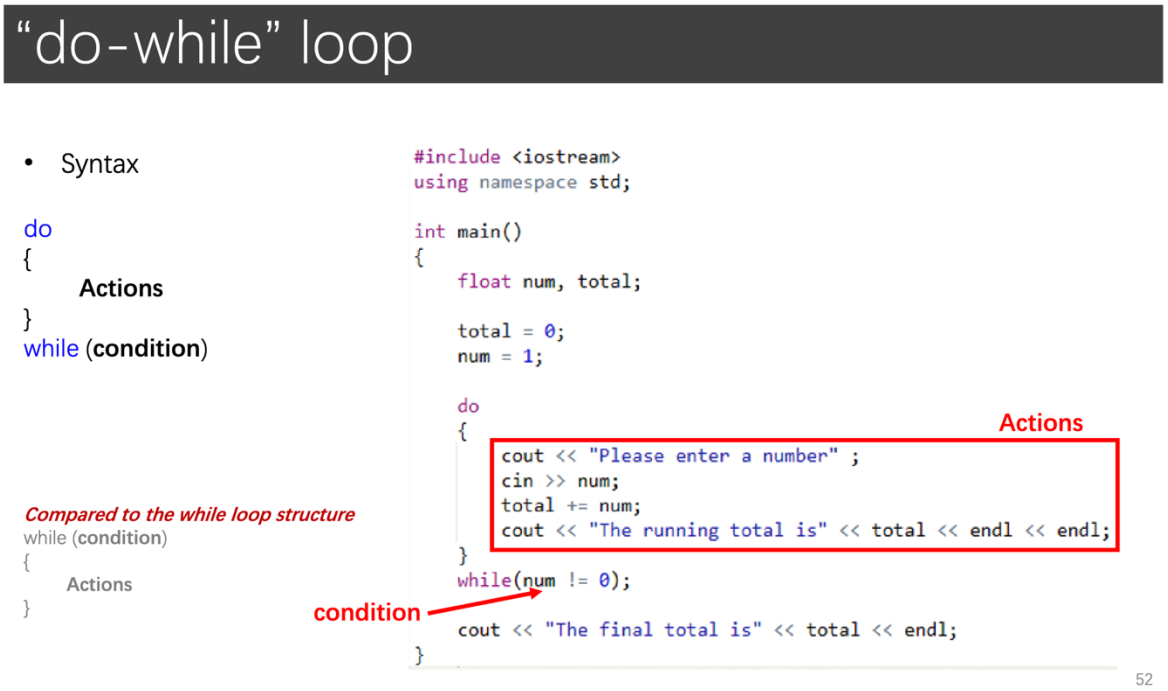
condition

Actions

```
10, 9, 8, 7, 6, 5, 4, 3, 2, 1, liftoff!
```

`do-while` Loop:** Evaluates condition **after** executing body (guarantees at least 1 execution).

do { /* body */ } while (condition);



LECTURE 6: ARRAYS AND STRUCTURES

6.1 One-Dimensional Arrays

Definition: A fixed-size collection of elements of the same data type stored in contiguous memory locations.

Declaration: `int numbers[10];` (Index ranges from `0` to `N-1`).

Initialization:

int days[5] = {31, 28, 31, 30, 31};

float values[5] = {2.3, 5.8}; // Unspecified elements auto-initialized to 0

int nums[] = {1, 2, 3}; // Automatically sets size to 3

6.2 Multi-Dimensional Arrays

Two-Dimensional Array: Matrix structure with rows and columns.

int usage[7][5]; // 7 rows, 5 columns

Initialization:

int vals[2][3] = { {4, 9, 5}, {2, 11, 3} };

int vals[][3] = { {4, 9, 5}, {2, 11, 3} }; // First dimension can be omitted

Example: reads in the number of students using the five laboratories over seven days

```
4  #include <iostream>
5  using namespace std ;
6
7  main()
8  {
9    const int NO_OF_DAYS = 7 ;
10   const int  NO_OF_LABS = 5 ;
11   int usage[NO_OF_DAYS][NO_OF_LABS] ;
12   int day, lab, total_usage, average ;
13
14   // Read each lab's usage for each day.
15   for ( day = 0 ; day < NO_OF_DAYS ; day++ )
16   {
17     cout << "Enter the usage for day " << ( day + 1 ) << endl ;
18     for ( lab = 0 ; lab < NO_OF_LABS ; lab++ )
```

Symbolic constant NO_OF_DAYS and NO_OF_LABS.

Define a two-dimensional array usage with NO_OF_DAYS rows and NO_OF_LABS columns.

Example: reads in the number of students using the five laboratories over seven days

```
14   // Read each lab's usage for each day.
15   for ( day = 0 ; day < NO_OF_DAYS ; day++ )
16   {
17     cout << "Enter the usage for day " << ( day + 1 ) << endl ;
18     for ( lab = 0 ; lab < NO_OF_LABS ; lab++ )
19     {
20       cout << " Lab number " << ( lab + 1 ) << ' ' ;
21       cin >> usage[day][lab] ;
22     }
23   }
24
```

The for loop reads in values into the array **usage[day][lab]**

row and column index

6.3 Structures (`struct`)

Definition: A user-defined data type grouping heterogeneous (different) data types into a single unit.

Template Declaration:

```
struct Student {
  int number;
  float scores[5];
};  // Semicolon is mandatory!
```

Variable Creation & Access: Access members using the dot (`.`) member selection operator.

```
Student s1 = {1234, {90, 85, 88, 92, 95}};
s1.number = 5678;
```

Nested Structures: Structures contained within other structures.

```
person.joined.year = 2026;
```

Note: Structure variables cannot be compared directly using `==` (e.g., `s1 == s2` is invalid); each member must be compared individually.

6.4 Enumerated Data Types (`enum`)

User-defined data type representing a set of named integer constants:

```
enum Response { no = 0, yes = 1, none = 2 };
Response answer = yes;
```

6.5 Searching & Sorting Algorithms

-Bubble Sort (Sort by Exchange): Continuously compares adjacent elements and swaps them if out of order.

-Selection Sort: Finds the minimum element from the unsorted portion and swaps it with the first unsorted position.

-Linear Search: Scans elements sequentially from start to end.

-Binary Search: Repeatedly divides a sorted array in half to locate the target value efficiently.

Quick syntax reference

	Syntax	Examples
Defining arrays	`type array[d1][d2]...[dn] ;` Dimensions `d1,d2...dn` are integer constants.	`int a[10] ;` `float b[5][9] ;`

Array subscripts	`array[i1][i2]...[in]`
	indexes or subscripts `i1,i2...in` are integer constants or variables.
		`a[0] // 1st element.`
		`a[9] // 10th element.`
		`b[0][0] // Row 1, col 1.`
		`b[4][8] // Row 5, col 9.`

Declaring a structure template	`struct structure_tag`
	`{`
	`  type variable1 ;`
	`  type variable2 ;`
	`  ...`
	`} ;`
		`struct date`
		`{`
		`  int day ;`
		`  int month ;`
		`  int year ;`
		`} ;`

Defining structure variables	`struct structure_tag` $variable_1$, $variable_2$, ... ;	`struct date dob ;`
Accessing structure members	Member selection operator. (Dot operator)	`dob.day ;`

LECTURE 7 STRING

2 main type :

- C - STRINGS
- C++ STRINGS

7.1 C - STRINGS

Definition : array of characters with the null character '\0' in the last element of array

//So always add +1 of the actual length of the string

Syntax :

char greetings[6] = {'H', 'E', 'L', 'L', 'O', '\0'};

//hello is 5 character but the length of char becomes 6

- String literal

use double quotation

char greetings[ ] = {"Hello"};

//So we don't need to specify the length, compiler automatically adds a null.

7.2 C - STRINGS input and output

```
4  #include <iostream>
5  using namespace std ;
6
7  main()
8  {
9    const int MAX_CHARACTERS = 10 ;
10   char first_name[ MAX_CHARACTERS + 1 ] ;
11
12   cout << "Enter your first name (maximum "
13        << MAX_CHARACTERS << " characters) " ;
14   cin >> first_name ;
15   cout << "Hello " << first_name << endl ;
16 }
```

//Remember to +1 the length of array

//The users name already stored in the array

//cin only reads until whitespace

Ex :

Input : John Paul

Output : Hello John

//What if the user press [space]?

//What if the user type more than 10 characters?

- .getline( ) function

Definition : Read input until the length is reached or deleimiter is pressed

//even tho there's space, ex : John Paul, the output still John Paul

Syntax :

.getline(*array, length, delimiter*)

//If delimiter left blank, it's usually '\n' (enter)

```cpp
#include <iostream>
using namespace std ;

main()
{
  const int MAX_CHARACTERS = 10 ;
  char first_name[ MAX_CHARACTERS + 1 ] ;

  cout << "Enter your first name(maximum "
       << MAX_CHARACTERS << " characters) " ;
  cin.getline( first_name, MAX_CHARACTERS + 1, '\n' ) ;
  cout << "Hello " << first_name << endl ;
}
```

Same number as the dimension of the array.

- cin.ignore(80'\n')



7.2 C - STRINGS Function

//Don't forget to use
#include <cstring>
Functions :

- strlen(str)
- strcpy(destination, source)
- strcat(str1, str2)
- strcmp(str1, str2)

Exaples :

strlen(str)

```
char name1[]   = "Sharon" ;
char name2[10] = "Mark" ;
int len ;
len = strlen( name1 ) ;
cout << setw( 3 ) << strlen( name1 )
     << setw( 3 ) << strlen( name2 )
     << setw( 3 ) << strlen( "Rob" )
     << setw( 3 ) << len ;
```

```
Output:
6
4
3
6
```

String Length → Returns **Number of Characters** in a string *excluding \0*

strcpy(dest, source)

```
char name1[] = "Sharon" ;
char name2[10] = "Mark" ;
// Copy the contents of name1 to name2.
strcpy( name2, name1 ) ;
// Restore the original name.
strcpy( name2, "Mark" ) ;
```

String Copy → Copies the contents of the **source** string to **destination** string

strcat(str1, str2)

```
char str1[15] = "first & " ;
char str2[] = "second" ;
strcat( str1, str2 ) ; // str1 is now "first & second".
```

String Concatenate → **Concatenates** 2 strings (adds them up) and stores them into str1.

Enough memory must be allocated to str1 to hold the result of the concatenation.

strcmp(str1, str2)

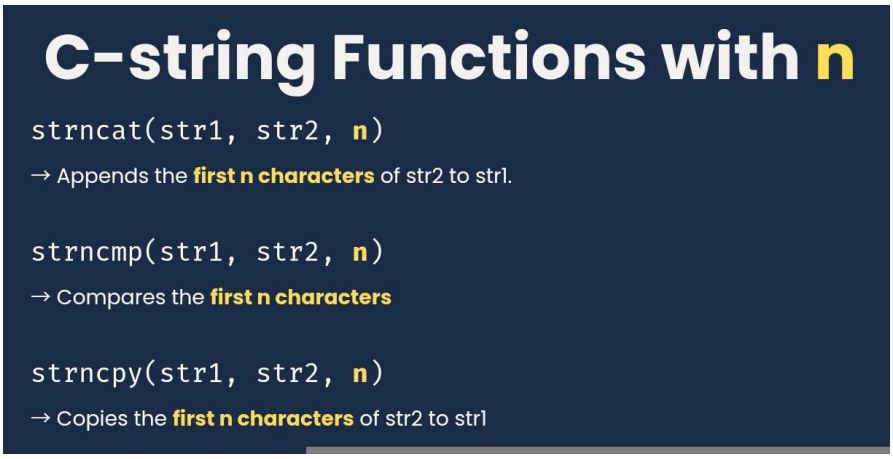
```
3  #include <iostream>
4  #include <cstring>
5  using namespace std ;
6
7  main()
8  {
9    char password[7] = "secret" ;
10   char user_input[81] ;
11   cout << "Enter Password: " ;
12   cin >>  user_input ;
13   if ( strcmp( password, user_input ) == 0 )
14     cout << "Correct password. Welcome to the system ..." << endl ;
15   else
16     cout << "Invalid password" << endl ;
17 }
```

This **Function** Returns:

- a **negative value**, if str1 < str2
- **0**, if str1 == str2
- a **positive value**, if str1 > str2

→ Usually used to compare if 2 strings have the same exact contents

Compared by its **ASCII value**.
ex: *"apple" < "banana"* because '**a**' (ASCII 97) is **less than** '**b**' (ASCII 98).



Summary C-string Functions

Function	Remark
strlen(str)	Finding the length of a C-string str
strcpy(str1, str2)	copies the contents of a C-string str2 to str1
strcat(str1, str2)	concatenates a C-string str2 to the end of the C-string str1
strcmp(str1, str2)	compares two null-terminated C-strings str1 and str2.
strncat(str1, str2, n)	Appends the first n characters of the C-string str2 to str1.
strncmp(str1, str2, n)	Identical to strcmp, except that at most n characters are compared.
strncpy(str1, str2, n)	Copies the first n characters of str2 into str1.

7.3 C++ strings

//Don't forget to use
#include <string>
Definition : used to store and manipulate sequences of characters
//string is not built into C++, unlike int or double (need to use #include <string>)
Why use c++ strings :

- Automatically handles memory management (don't need to define array size)
- Hidden null terminators.
- Safer and Easier to work with.

Syntax :
Example

```
string str1 = "ABCDEFG";
string str2(11, '-');          // creates a string of 11 dashes "----------"
string str3   =  "This is the first part,"
                 " and this is the second part";
```

7.4 C++ strings function

Functions :

- str.assign(param)
- str.swap(target)
- Concatenate Strings (+)
- str.append(param)
- str.length()
- str.substr(param)
- String Searching
- String Comparison
- str.replace(param)
- str.erase(param)
- str.insert(param)

//Learn how each function work

LECTURE 8 FUNCTIONS

8.1 Functions

Definition :Blocks of **statements** called by **name** to carry out a **specific task**

//When u want to do operations repeatedly with a formula u have made, but don't want to rewrite the operation from the start again and again, u can just **call the name of the function**

Syntax :

- **void** function_name (**void**) //returns nothing, receive nothing

//When this kind of function is called, it doesn't return anything and doesn't need to input any parameters, usually a cout function

- **int** function_name (**int**) //returns integer, receive integer

//When this kind of function is called, it returns an integer for example '123', and it needs an integer parameter

Example :

Returns **Integer** Receives **Integer**

```
int minimum_val(int num1, int num2){
  if (num1 > num2){
     return num2;
  } else {
     return num1;
  }
}

int main(){
  int val1 = 0;
  int val2 = 1;
  cout << "the minimum of " << val1 << "and" << val2
       << "is" << minimum_val(val1,val2) << endl;
}
```

//it receives **int** val1 and **int** val2, and returns **int** val2

Structure :

Function header

Function body

```
3  #include <iostream>
4  #include <string>
5  using namespace std;
6
7  void stars( void );
8
9  int main()
10 {
11     string text = "some text";
12
13     stars();   //Call the function to display the top of the box
14     cout << endl;
15     cout << "*";    //Left side of the box
16     cout << text;   //Text in middle of the box
17     cout << "*" <<endl; //Right side of the box
18     stars();
19     cout << endl;
20 }
21
22 void stars( void )
23 {
24     for( int counter = 0 ; counter < 11 ; counter++ )
25     cout << '*';
26 }
```

Line 22 **Function header**
Line 24-25 **Function body**

8.2 Function arguments

- **Argument** Definition : Argument is actual value that is put in the parameter when calling the function

Stars(**11**) //11 is argument

- **Parameter** Definition : Variable inside the function that receives the value (argument)

void stars (**int num**) //int num is parameter

- Default values

Definition : a parameter that is not a passed value

Syntax :

```
void disp_chars(int num = 1, char ch) ;×
void disp_chars(int num, char ch = ' ') ;√
```

Function uses **Default Value** if **nothing** is passed when called

If you set a default value in the parameter, then the parameters **TO THE RIGHT must** also have a default value

8.3 Passing arguments

2 ways :

- Pass by Value

Definition : pass by value the creates a copy of the original variable(argument) and hand the copy to the function's parameter

//It's like handing someone a photocopy of a document. If they scribble on the copy, your original paper at home is fine.

Example :

```
6  void any_function ( int p );
7
8  main()
9  {
10     int a = 1;
11     cout << "a is" << a << endl ;
12
13     any_function (a) ;
14
15     cout << "a is still " << a << endl  ;
16 }
17
18 void any_function( int p )
19 {
20     cout << "p is" << p << endl ;
21     p = 2 ;
22 }
```

A copy of the value of *a* is passed to *p*

Changing the value of *p* has no effect on *a*

Output :

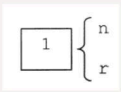
```
a is 1,
p is 1,
a is still 1
```

- Pass by reference

Definition : **pass by reference**, you don't send a copy. Instead, you use an ampersand (&) to give the function a direct link (an alias) to the **original variable**.

```
int n = 1;
int& r = n;

n = 2;      // changes both n and r
```



- **r** is a **reference** to **n**
- **n** and **r** both refer to the **same value**
- **r** is not a copy of **n**, but is merely **another name** for **n**
- A **change** to **n** will also result in a **change** to **r**

Example

```
6  void any_function ( int& p );
7
8  main()
9  {
10     int a = 1;
11     cout << "a is" << a << endl ;
12
13     any_function (a) ;
14
15     cout << "a is now " << a << endl  ;
16 }
17
18 void any_function( int& p )
19 {
20     cout << "p is" << p << endl ;
21     p = 2 ;
22 }
```

a and **p** refer to the **same Storage Location.**
Changing the value of **p** changes also the value of **a.**

Output:

```
a is 1
p is 1
a is now 2
```

Passing an Array

→ Can only be passed by reference to a function

//So passing an array **doesn't need** the '**&**'

Example

```
11 int main()
12 {
13     int values[10] = { 12, 4, 5, 3, 4, 0, 1, 8, 2, 3 } ;
14     int sum ;
15
16     sum = sum_array( values , 10 ) ;
17     cout << "The sum of the array elements is" << sum <<endl;
18 }
19
20 int sum_array ( const int array [], int no_of_elements)
21 {
22     int total = 0;
23
24     for( int index = 0 ; index < no_of_elements ; index++ )
25         total += array[index] ;
26     return total ;
27 }
```

WHAT DOES THIS MEAN?

- If we modify the array[], it will modify the ***values[10]***
- keyword **const**: array cannot be changed, thus protecting the ***values*** array.
- [] are necessary to indicate that the parameter **is a reference to an array.**

The *number of elements* is **not required** in the brackets for a *one-dimensional array* to handle different size array.

Passing a Multi-dimensional Array

```
int sum_array( const int array[][2], int no_of_rows)
{
    int total=0;

    for (int row =0; row < no_of_rows ;row++)
    {
        for( int col=0;col <2;col++)
            total += array[row][col];
    }
    return total;
}

main()
{
    int values[5][2] = { { 31, 14 },
                         { 51, 11 },
                         {  7, 10 },
                         { 13, 41 },
                         { 16, 18 } };
    int sum;

    sum= sum_array(values,5);
    cout<< "The sum of the array elements is " << sum << endl;
}
```

Output:

```
The sum of the array elements is 212
```

The Array parameter **MUST** contain the **DIMENSION SIZE** other than the first slot.

8.4 Mathematical functions

#include<cmath>

- Trigonometric functions

```cpp
#include <cmath>
using namespace std ;

main()
{
    const double RADIANS_IN_A_DEGREE = 57.29578 ;

    double degrees, radians ;

    cout << "Input the angle in degrees:" ;
    cin >> degrees ;
    radians = degrees / RADIANS_IN_A_DEGREE ;
    cout << fixed << setprecision( 3 )
         << "sin(" << degrees << ")=" << sin(radians) << endl
         << "cos(" << degrees << ")=" << cos(radians) << endl
         << "tan(" << degrees << ")=" << tan(radians) << endl ;
}
```

Output:

```
Input the angle in degrees: 60
sin(60.000)= 0.866
cos(60.000)= 0.500
tan(60.000)= 1.732
```

- Pseudo-random number functions

rand()

srand()

```cpp
#include <cstdlib> // Required for rand() and srand()
#include <ctime>   // Required for time()

srand(time(0));    // time(0) gets the current system time in seconds

// rand() % 10 gives a number from 0 to 9. Adding 1 shifts it to 1 to 10.
int ranged_num = (rand() % 10) + 1;

// Generate a random number between 50 and 100
// Formula: (rand() % (max - min + 1)) + min
int custom_range_num = (rand() % (100 - 50 + 1)) + 50;
```

→ **srand()** generates **a seed** for the random number generator.
time(0) ensures that the seed is always **unique**.
→ **rand()** generates a random number.

8.5 Function Overloading

Definition : feature of C++ that allows you to give the **exact same name** to two or more **different functions**, as long as they perform similar tasks but **take different arguments** (either a different number of parameters or different data types).

//The c++ can tell the function apart by itself, it automatically choose which function to use based on the parameter of the function

Example :

Addition program

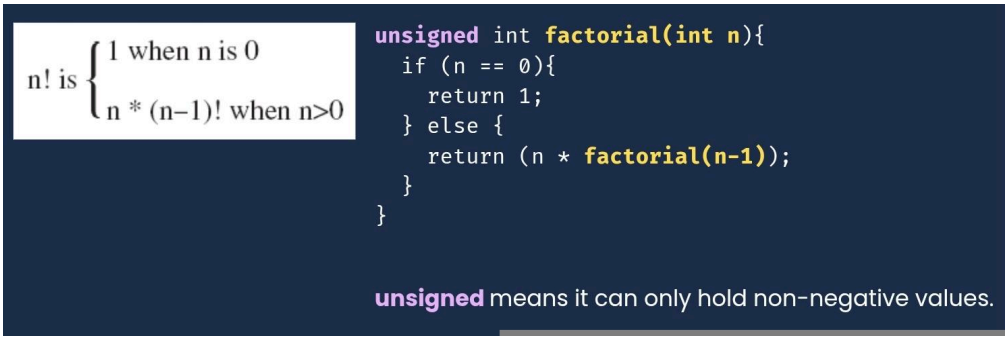
```cpp
int add(int x, int y);              // Version 1: takes two integers
int add(float x, float y);          // Version 2: takes two floats
int add(int x, int y, int z);       // Version 3: takes three integers
```

→ If you call add(5, 10), C++ automatically runs Version 1 because you passed two integers.
→ If you call add(5.5f, 2.1f), C++ runs Version 2 because you passed floats.
→ If you call add(1, 2, 3), C++ runs Version 3 because you passed three arguments.

8.6 Recursion

Definition : programming technique where we call a function inside its own function

Example :



8.7 The scope of a variable

Definition : specific region of a program where the variable is defined

- Block scope

Definition : **Block** is statements enclosed in **curly braces { }**.

Variables defined in a block is accessible **only within** that block.

//As soon as the program exits that block (hits the closing }), the variable is destroyed and erased from the memory

- Loop scope

Definition : declare a variable directly **inside** the **initialization part of a for loop**

(e.g., for(int i = 0; i < 10; i++))

//that variable i only exists inside that loop. Once the loop finishes, i is destroyed and cannot be used anymore.

- Global scope

Definition : variable declared outside of all functions (usually at the very top of your file, **outside main()**) is a global variable.

Can be accessed and modified by any function anywhere in the program

LECTURE 9(a) OBJECTS AND CLASSES

9.1 Basics of objects and class

- Object

Definition : a component of a program that knows

- How to perform certain actions
- How to interact with other parts of the program

Consists of :

- Data values
- Functions

- Class

Definition : Class is general category defines the characteristics and functions for object of that category

Consists of :

a. **Private**→ Characteristics, called properties or class data members
   //for example the class is animal, the properties could be :
   - Name
   - Species
   - age

b. **Public**→ Functions, called class member functions or method
   usually contains :
   - **Constructor**
   - **Class method**

The difference between class and objects :

→ class is a template that can be used to create many instances of the class
→ instance of a class is the actual object created from the template
//Example : class is like a a noun, and object is proper noun
person is a noun, John smith is a proper noun
Person is the class and John smith is the object

9.2 Constructing a class in C++

A. PRIVATE

Example :

```
class Animal {
  // Private members can only be accessed from within the class itself
  private:
    string name;
    string species;
    unsigned int age; // Using unsigned since age cannot be negative


  Private usually contains member Data.
  → Can only be accessed by member Functions of the class
```

B. PUBLIC

- Constructor

Definition : is a **Function** that **runs automatically** when an instance of the class is created. Has the **same name** as the class, **no return** type.

Example :

```
public:
    Animal(string animalName, string animalSpecies, unsigned int animalAge) {
        name = animalName;
        species = animalSpecies;
        age = animalAge;
    }
```

Used to **Initialize** the values of the **Data Members**
→ by ***passing arguments*** through the class' ***parameters***.

Deafult class constructor → constructor that has no parameters

Example :

bank_account ();

- Class Method (member function)

Definition : Basically a function

Example :

```
public:
    // Member Function (Method): Defines an action the object can perform
    void makeSound(string sound) {
        cout << name << " the " << species << " says: " << sound << "!" << endl;
    }

    void displayInfo() {
        cout << "Name: " << name << ", Species: " << species << ", Age: " << age
<< " years old." << endl;
    }
```

Public usually contains **member Functions** and the **Constructor**.
→ Can **accessed** from **outside** the class.

Through the **Member Functions**, we can modify the **Member Data**

Creating Objects

Example :

//The class is animal, the object (class instances) we create is cat, dog, using the animal class template

```
    // We pass the starting values to the constructor
    Animal cat("Luna", "Cat", 3);
    Animal dog("Rex", "Dog", 5);

    // Calling the member functions using the dot (.) operator
    cat.displayInfo();
    cat.makeSound("Meow");

    dog.displayInfo();
    dog.makeSound("Woof");
```

The **Animal class** is just the **template**
cat and **dog** are the actual *objects (instances)* created using that template.
→ They each hold their own unique set of name, species, and age data.

Overloading Constructors

//Same as function, constructor can also be overloaded

Example :

```
// Used when no information is provided at creation.
    Animal() {
        name = "Unknown";
        species = "Unknown";
        age = 0;
    }
// Used when name and species are known, but we want to set a default age.
    Animal(string animalName, string animalSpecies) {
        name = animalName;
        species = animalSpecies;
        age = 1; // Defaulting to 1 year old
    }
```

Constructor is a **function** (can be **Overloaded**)
→ We can pick which contructor to use **depending** on the **parameters** passed on

9.4 Class properties

//Here is the example we'll use

```
int Animal::totalAnimals = 0;              // initialize static property OUTSIDE
class Animal {
  private:
    string name;

    // This variable is shared across all Animal instances
    static int totalAnimals;

  public:
    Animal(string animalName) {.           // CONSTRUCTOR - always called
        name = animalName;
        totalAnimals++;                    // runs when new object is created
    }

    // A static method can only access static variables.
    // It can be called without needing a specific object.
    static int getTotalAnimals() {
        return totalAnimals;
    }
};
```

```
// Create a few animals
    Animal cat("Luna");
    Animal dog("Rex");
    Animal bird("Tweety");

// The shared counter has now been incremented 3 times
    cout << "Count after creating objects: " << Animal::getTotalAnimals() << endl;
```

- Static Property

Definition : A **variable** declared with the keyword '**static**' **inside a class**

Only **one copy** of a static data member exists and is shared by all objects of a particular class. → Shared Memory

Syntax :

Ex : `static int totalAnimals;`

Initialization Rule

- Static data members initialized **outside the class** and **outside main()**

Syntax :

Ex : `int Animal::totalAnimals = 0;`

//The double '**::**' operator means we're accessing something inside the class.

Animal::totalAnimals → accessing the totalAnimal item inside the animal class

- Inspector and Mutator Functions

Inspector

Definition : allow private data members of a class to be inspected from outside the class

Syntax :

Ex :
```
static int getTotalAnimals() {
        return totalAnimals;
    }
//Usually it's just a function whose job is to read and return a value
```

Mutator

Definition : modify values of private data members

- Inline functions use inline
- Splitting code into header and source files
  Header use (.h)
  Source file use (.cpp)

LECTURE 9(b) CLASS INHERITANCE

9.1 Inheritance and its syntax

Definition : allows a **new class** to be constructed using an **existing class** as a basis.

- Existing class is called **base class**
- New class is called **derived** or **inherited class**

Base Class

```
class Animal {
// 'protected' means these variables are hidden from outside (like 'private'),
// BUT they are accessible to any derived child classes.
  protected:
    string name;
    int age;

  public:
    Animal(string animalName, int animalAge) {
    name = animalName;
    age = animalAge;
    }

    void eat() {
      cout << name << " is eating." << endl;
    }
};
```

Derived Class

```
class Dog : public Animal {
  private:
   string breed;

  public:
// The derived class constructor MUST pass the data to the base class constructor.
// using an initialization list: : Animal(dogName, dogAge)
    Dog(string dogName, int dogAge, string dogBreed) : Animal(dogName, dogAge) {
     breed = dogBreed; // Initialize the Dog-specific property
    }

    void bark() {
// We can use 'name' here directly because it is 'protected' in the Animal class
      cout << name << " (a " << breed << ") says: Woof!" << endl;
    }
};
```

Create Objects

Base Class 'protected' data — **Derived Class** 'private' data

```
Dog myDog("Rex", 5, "Golden Retriever");
// Using inherited methods from the Animal base class
myDog.eat();

// Using specific methods from the derived classes
myDog.bark();
```

Override Methods

```
class Dog : public Animal {
  private:
   string breed;

  public:
    Dog(string dogName, int dogAge, string dogBreed) : Animal(dogName, dogAge) {
     breed = dogBreed;
    }

    void eat() {   // The same eat() function is being overriden in the Dog class
      cout << name << “is eating dog food” << endl;
    }
};
```

Types of Inheritance

Type of inheritance	Base class member access	Derived class member access
public	public	public
	protected	protected
	private	inaccessible
protected	public	protected
	protected	protected
	private	inaccessible
private	public	private
	protected	private
	private	inaccessible

class **Dog** : **public Animal**{}
class **Dog** : **protected Animal**{}
class **Dog** : **private Animal**{}

LECTURE 10 POINTERS AND MEMORIES

10.1 Addresses and Pointers

- Address

Definition : every variable and object used in C++ program is stored in a specific place in **memory** → each **location** in the memory has a unique **address**

Syntax :

Use '&' to get the address of a variable

&var

- Pointers

Definition : A **pointer variable** is a variable that holds the **address of another variable**

Syntax :

Use '*' to get the address of a variable

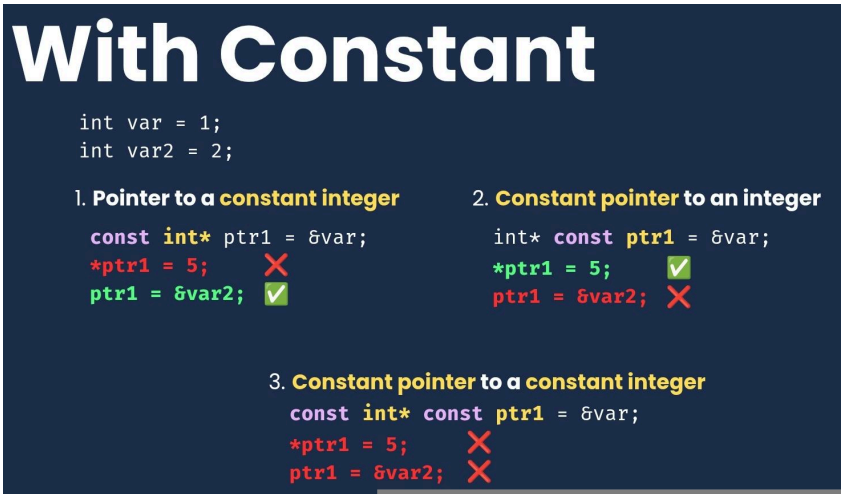
data_type* variable_name;

Example:

```
int var = 1;       // A standard integer variable.
int* ptr;          // ptr is a pointer to an int variable[cite: 88, 89].

ptr = &var;        // ptr now contains the memory address of var. (0012FF88)

cout << &var << endl; // e.g., 0012FF88
cout << ptr << endl;  // e.g., 0012FF88
cout << *ptr << endl;  // e.g., 1
```



10.2 Pointers and Arrays

Definition : Array itself is a pointer to the first element of the array.

Use * to access the elements of the array

```
int a[5] = {1, 2, 3, 4, 5};
```

The **name** of the array **a** itself is a **pointer** to the first index

```
  a   → 0012FF78
&a[0] → 0012FF78
```

So we can **dereference** with * to access the elements

```
*(a + 1) → a[1]
*(a + 2) → a[2]
```

but array pointers are constant

❌
```
a++
a = a + 1
```

✅
```
int* p = a;   // initialize p as a pointer
p++;          // → a[1]
```

- Multi-Dimensional arrays

```
int a[3][2] = {
        {4, 6},
        {1, 3},
        {9, 7}
    };
```



int** **int***

```
*a → {4,6}          **a   → a[0][0]      *(a[0]+1) → a[0][1]
*(a+1) → {1,3}      *a[1] → a[1][0]      *(a[1]+1) → a[1][1]
*(a+2) → {9,7}      *a[2] → a[2][0]      *(a[2]+1) → a[2][1]
```

Each row | **First Element** of each row | **+1 Element** of each row

10.3 Pointers to classes

Definition : Access **data** and **functions** of Classes and Structures.

Syntax :

Structure pointer format

```
Struct tag_name* variable_name ;
```

Class pointer format

```
class_name* variable_name ;
```

Access **data** and **functions** of Classes and Structures.

```
class Animal {
private:
    string species;
    int age;
public:
    Animal(string sp, int a) {
        species = sp;
        age = a;
    }
    void have_birthday() {
        age++;
    }
}
```

```
Animal my_pet("Golden Retriever", 3);

Animal* pet_ptr;
pet_ptr = &my_pet;

pet_ptr->have_birthday();
(*pet_ptr).have_birthday();
```

Accessing members using a pointer
2 ways :

- Using dereference operator (*)

Definition : **ptr** → holds an **address**, so ***ptr** gets the **actual object**
Syntax : `(*pet_ptr).have_birthday();`

- Using arrow notation (->)

Definition : use -> to avoid parentheses ( )
Syntax : `pet_ptr->have_birthday();`

10.4 Pointers as function arguments

Definition : Passing a memory address of variables into a function using pointers

Using pointers as Functions Arguments

```
void swap_vals(float* ptr1, float* ptr2) {
    float temp = *ptr1;
    *ptr1 = *ptr2;
    *ptr2 = temp;
}

float num1 = 12.1;
float num2 = 6.4;

swap_vals(&num1, &num2);
```

10.5 Dynamic Memory Allocation

→ To make sense of it, the **new** keyword allows us to set the size of the array when the program is running. Usually, we set the length during coding, but if we want to set it depending on, let's say, the user input, then we'll need the **new** keyword

DON'T FORGET → **delete[] variable;**
If not, then there will be memory leaks.

- When **allocating memory for an array of class objects**, there must be a **default constructor for the class** so that the elements of the array get initialized.

```
bank_account*  ac_ptr ;

ac_ptr = new back_account[5] ;
```

Default constructor of the "bank_account" class being called five times

We can also use the **new** keyword for creating other variables (not array only)

```
// 1. Allocating a single primitive variable
// Allocates memory for one integer and initializes it to 42
int* myNumber = new int(42);
```

cerr is used to inserting error messages into the output stream
→ behaves exactly like a cout:

cerr << "error message" << endl;

LECTURE 11 POLYMORPHISM

11.1 Polymorphism

```
6  class advanced_computer
7  {
8  public:
9  void hello()
10 {
11   cout << "Hello from the Advanced Computer" << endl ;
12 }
13 } ;
14
15 class simple_computer
16 {
17 public:
18 void hello()
19 {
20   cout << "Hello from the Simple Computer" << endl ;
21 }
22 } ;
23
```

```
24 main()
25 {
26   advanced_computer HAL ;
27   simple_computer PC ;
28   HAL.hello() ;
29   PC.hello() ;
30 }
```

Polymorphism:
The same command to different objects invokes different actions.

11.2 Static and Dynamic Polymorphism

- Function overloading
- Operator overloading
- Templates

One example of polymorphism is Overloading a function. Function overloads have the same function NAME, but behaves differently depending on the parameters passed on.

function overloading

```
int sum_array ( const: int array [] , int no_of_elements )
{
    int total = 0;

    for(int index = 0 ; index < no_of_elements ; index ++ )
        total += array[index] ;
    return total;
}

int sum_array ( const int array[][2] int no_of_rows )
{
    int total = 0;

    for(int row = 0 ; index < no_of_rows ; row ++ )
    {
        for(int col = 0 ; col < 2 ; col++ )
            total += array[row][col] ;
    }
    return total;
}
```

Functions with the same names and different parameter lists mean function overloading.

Operator Overloading:

When we want to overload Operators, the function's name starts with "operator" and then the symbol we want to overload.

The example below is the overloading of the + operator, where we overload it so that we can combine 2 instance of a class together to get their total points.

```cpp
#include <iostream>

class Score {
public:
    int points;

    // Simple constructor to set the points
    Score(int p) {
        points = p;
    }

    // The simplest operator overload for '+'
    Score operator+(Score other) {
        // 1. Add this object's points with the other object's points
        int totalPoints = points + other.points;

        // 2. Create and return a brand new Score object with the combined total
        return Score(totalPoints);
    }
};
```

```cpp
int main() {
    Score game1(10);
    Score game2(5);

    // We can now use '+' directly on our custom Score objects!
    // Behind the scenes, game1 is calling the function, and game2 is passed as 'oth
    Score total = game1 + game2;

    std::cout << "Total points: " << total.points << "\n"; // Outputs 15

    return 0;
}
```

11.3 Template

```cpp
template <typename T>
T maximum( const T n1, const T n2
{
  if ( n1 > n2 )
    return n1 ;
  else
    return n2 ;
}
```

```
6   template <typename T>
15 main()
16 {
17   char c1 = 'a', c2 = 'b' ;
18   int i1 = 1, i2 = 2 ;
19   float f1 = 2.5, f2 = 3.5 ;
20
21   cout << maximum( c1, c2 ) << endl ; //  a maximum() for chars
22   cout << maximum( i1, i2 ) << endl ; //  a maximum() for ints
23   cout << maximum( f1, f2 ) << endl ; //  a maximum() for floats
24 }
```

This process is called *instantiation of the template* and the result is *a* conventional function generated by the compiler.

```
template <typename T>
T maximum( const T n1, const T n2
{
  if ( n1 > n2 )
    return n1 ;
  else
    return n2 ;
}
```

Line 21 has the effect of replacing the type parameter *T* in line 6 with the data type of the arguments c1 and c2, i.e. char. The compiler generates a function with the prototype.

Templates can also be used for Classes.

Class Templates Example

```
#include <iostream>
using namespace std;

template <class T>
class Number {
  private:
   // Variable of type T
   T num;

  public:
   // constructor
   Number(T n) : num(n) {}

   T getNum() {
      return num;
   }
};

int main() {

   // create object with int type
   Number<int> numberInt(7);

   // create object with double type
   Number<double> numberDouble(7.7);

   cout << "int Number = " << numberInt.getNum() << endl;
   cout << "double Number = " << numberDouble.getNum() << endl;

   return 0;
}
```

Ex. → Created a class called “numberInt” with the template data type of <int>, passing the value of 7 to the constructor.

11.4 Virtual Functions

Virtual is an example of Dynamic Binding.
The virtual keyword is actually optional in Derived Classes, but it’s a good idea to include it for the sake of clarity.

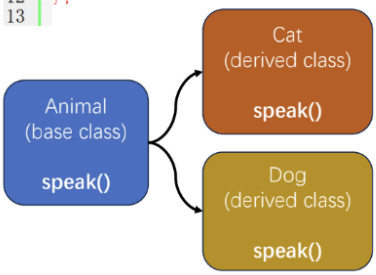
First Example DOESN’T USE the **virtual** keyword

```cpp
#include <iostream>
#include <string>

using namespace std;

class Animal
{
public:
    Animal() {};

    string speak() { return "???"; }
};
```




```cpp
class Cat: public Animal
{
public:
    Cat() {};

    string speak() { return "Meow"; }
};

class Dog: public Animal
{
public:
    Dog() {};

    string speak() { return "Woof"; }
};

void Says(Animal& animal)
{
    cout << animal.speak() << '\n';
}

int main()
{
    Animal myAnimal;
    Cat myCat;
    Dog myDog;

    cout << "My animal (base class) says ";
    Says(myAnimal);

    cout << "My cat says ";
    Says(myCat);

    cout << "My dog says ";
    Says(myDog);
}
```

Code execution result:

```
My animal (base class) says ???
My cat says ???
My dog says ???
```

Now this is how to implement the **virtual** keyword on the speak() function:

```cpp
class Animal
{
public:
    Animal() {};

    virtual string speak() { return "???"; }
};

class Cat: public Animal
{
public:
    Cat() {};

    string speak() { return "Meow"; }
};

class Dog: public Animal
{
public:
    Dog() {};

    string speak() { return "Woof"; }
};

void Says(Animal& animal)
{
    cout << animal.speak() << '\n';
}

int main()
{
    Animal myAnimal;
    Cat myCat;
    Dog myDog;

    cout << "My animal (base class) says ";
    Says(myAnimal);

    cout << "My cat says ";
    Says(myCat);

    cout << "My dog says ";
    Says(myDog);
}
```

Code execution result:

```
My animal (base class) says ???
My cat says Meow
My dog says Woof
```

11.5 Abstract Base Classes

→ A base class that contains ONLY VIRTUAL FUNCTIONS

- **Abstract base classes** are not used to create objects, but exist **solely as a base for deriving other classes**.

- It is **not** possible to define an object of an **abstract base class**, i.e. objects of an abstract base class **cannot be instantiated**.

```
class Animal    //Abstract base class
{
 public :

 virtual void speak() = 0; //Pure virtual Function
};
```

```
Animal myAnimal; // error : variable of an abstract class
```

The base class function has no code

→ virtual void speak() = 0;

Pure Virtual Function, only used to be overridden by the derived classes.

LECTURE 12 FILES AND STREAMS

12.1 Input/Output Class Hierarchy

ifstream → create input file objects (read the file)
ofstream → create output file objects (write in the file)
fstream → create file objects that can be used for both input and output.

#include

12.2 Basic I/O operations

REMEMBER #include <fstream>

- first create an instance of the appropriate class

```
ifstrem in ;      // in is an input file object
ofstream out ;    // out is an output file object
```

- After creating an instance of the appropriate class, the file object must be opened to associate it with a file stored on a hard disk or some other storage device.

```
in.open( "in.dat" ) ;     // in is associated with in.dat
out.open( "out.dat" ) ;   // out is associated with out.dat
```

- Creating an instance of a file object and opening a file can be combined into one statement by using the *ifstream* and *ofstream* class constructors.

```
ifstrem in( "in.dat" ) ;
ofstream out( "out.dat" ) ;
```

ifstream **in**(“in.dat”);
in → the Function Name
in.dat → the File Name

ifstream **in**(“in.dat”);
in → the Function Name
in.dat → the File Name

```
3  #include <iostream>
4  #include <fstream>
5  #include <string>
6  using namespace std ;
8  main()
9  {
10   char c = 'A' ;
11   int i = 1 ;
12   string s = "Hello" ;
14   ofstream out ;            // Create a file object out and
15   out.open( "file.txt" ) ;  // open file.txt
17   // Write some data to the file using <<
18   out << s << ' ' << c << ' ' << i << endl ;
19   out.close() ;  // Close the output file.
21   ifstream in( "file.txt" ) ;  // Use constructor to open the file.
22   // Read data from the file using >>
23   in >> s >> c >> i ;
24   in.close() ;  // Close the input file.
26   cout << "Data read from file:" ;
27   cout << s << ' ' << c << ' ' << i << endl ;
28 }
```

The function name that we specify for ofstream and ifstream behaves exaclty like cout and cin with the same operators.

out << string
in >> string

For ofstream, when we do **out.open(“file.txt”)**, if the file doesn’t exist, it automatically creates a new file with that name.

Member Function	Return Value
`fail()`	Returns the Boolean value `true` if the operation failed; otherwise the Boolean value `false` is returned.
`good()`	Returns the Boolean value `true` if the operation succeeded; otherwise the Boolean value `false` is returned. This is the opposite of `fail()`.
`eof()`	Returns the Boolean value `true` if the end of the file has been reached; otherwise the Boolean value `false` is returned.

```
out.open( "file.txt" ) ;
if ( out.fail() )  // Has the file failed to open?
{
  cerr << "Failure to open file.txt for output" << endl ;
  return 1 ;
}
```

```
ifstream in( "file.txt" ) ;
if ( in.fail() )
{
  cerr << "Failure to open file.txt for input" << endl ;
  return 1 ;
}
```

get() and put()

The get() and put() functions:

get() is associated to the in, and only reads ONE character
put() is associated to the out, writes only ONE character

```
in.get( c ) ; // Read the first character in the file.
count = 1 ;
while( !in.eof() ) // Loop while not end of file.
{
  out.put( c ) ;  // Write the character.
  in.get( c ) ;   // Read the next character.
  count++ ;
}
```

How to combine input and output command into a single command.
→ instead of out << and in >>, we can just do a single command “file” either ‘<<’ or ‘>>’ depending on either we want to read or write.

MEMORIZE THE SYNTAX

fstream function_name(“File.txt”, ios::in | ios::out)

```
ofstream out("file2.txt");

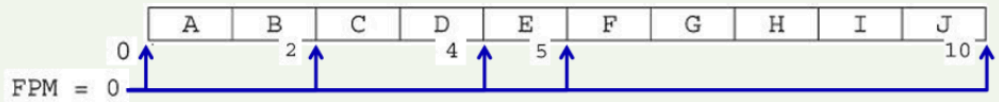
out << "Hello" << endl;
out << 12435 << endl;
out.close();


ifstream in("file2.txt");
string text;
int number;
string text2;
```

```
fstream file("file2.txt", ios::in | ios::out);
file << "C++Streams";
file >> text
```

Seekg() Function:

- Examples: Open a file containing the characters A to J with




```
istream in( “letters.txt” );
in.seekg( 4, ios::beg ) ;
in.seekg( 1, ios::cur ) ;
in.seekg( -3, ios::cur ) ;
in.seekg( 0, ios::beg ) ;
in.seekg( 0, ios::end ) ;
```

or `in.seekg( 4 ) ;`

FPM stands for File Position Marker

write() and read()

```cpp
// A simple struct holding mixed data types
struct PlayerSave {
    int level;
    float health;
    int gold;
};

int main() {
    // 1. WRITING BINARY DATA
    PlayerSave p1 = {15, 85.5f, 1500};

    // Open file in binary mode using std::ios::binary
    std::ofstream outFile("savegame.dat", std::ios::binary);

    if (outFile.is_open()) {
        // We cast the struct's address to a char*, and tell it exactly how many byt
        outFile.write((char*)&p1, sizeof(PlayerSave));
        outFile.close();
    }

    // 2. READING BINARY DATA
    PlayerSave p2; // Empty struct to hold the loaded data

    std::ifstream inFile("savegame.dat", std::ios::binary);

    if (inFile.is_open()) {
        // We cast the empty struct's address to a char*, and tell it how many bytes
        inFile.read((char*)&p2, sizeof(PlayerSave));
        inFile.close();
    }
```

write((char*)&p1, sizeof(class))
The Memory Address: Where in RAM is the data you want to save? Because these functions process raw bytes, you must cast the memory address of your data to a character pointer using `(char*)`.
The Size: Exactly how many bytes large is the data block? You use the `sizeof()` operator to figure this out.

Open the file using std::ios::binary → in binary mode
Usually this parameter is omitted.